

ENTREGABLE N.º 3

Diseño de Sistemas de Información

Residencia360 — Sistema de Información para la Unidad Residencial «Altos de Santa Clara»

Nombre del proyecto	Residencia360
Integrantes	Juan Camilo Cardona Sánchez · Yesid Ardila Elejalde
Docente	[Nombre del docente]
Curso	Diseño de Sistemas de Información
Fecha	Mayo de 2026
Institución	Instituto Tecnológico Metropolitano (ITM)
Repositorio Git	[pegar URL del repositorio Git]

Contenido

1. Propósito y relación con los Entregables N.º 1 y N.º 2

El Entregable N.º 3 evidencia la evolución técnica del proyecto Residencia360 a partir de la consolidación de los dos entregables anteriores. No es una actividad aislada, sino una fase de refinamiento, integración y validación funcional del sistema. En el Entregable N.º 1 se definió la base (modelo verbal, requisitos, reglas de negocio, story mapping, historias de usuario y diagrama de clases); en el Entregable N.º 2 esos insumos se desarrollaron técnicamente (refinamiento, arquitectura C4, patrones de diseño, prototipo funcional, diagramas de actividad y análisis SOLID).

Esta entrega demuestra que el sistema avanza hacia una solución más madura mediante: la modelación del comportamiento con máquinas de estado, la representación estructural y de distribución con un diagrama integrado de componentes y despliegue, la modelación de interacciones internas con diagramas de secuencia que incorporan flujos alternativos, y un MVP funcional de backend con validaciones y manejo de excepciones. Todos estos artefactos se derivan explícitamente de los requisitos, reglas e historias previos, garantizando trazabilidad y coherencia entre los tres entregables.

2. Refinamiento y consolidación de los entregables previos

Esta sección presenta los ajustes realizados sobre los artefactos de los Entregables N.º 1 y N.º 2 para alinearlos con la versión actual del sistema. No se repiten los entregables previos: se evidencia cómo fueron fortalecidos y conectados con el MVP, los diagramas UML de esta entrega y el repositorio Git.

Tabla 1. Ajustes realizados frente a los entregables anteriores.

Elemento ajustado	Origen	Cómo estaba antes	Cómo quedó refinado	Motivo
Estados de visita y PQRS	E1	Estados como texto suelto en las reglas (RN-02, RN-09).	Máquinas de estado UML formales con transiciones, guardas y acciones (§3).	Modelar comportamiento
Patrón State	E2	Justificado de forma teórica en la sección de patrones.	Especificado visualmente como máquinas de estado y reflejado en el backend.	Coherencia patrón-diagrama
Vista de componentes	E2	Niveles 2 y 3 del C4 por separado.	Diagrama integrado de componentes y despliegue con nodos y protocolos (§4).	Integrar estructura y distribución
Interacciones internas	E1/E2	Descritas en historias y actividades, sin secuencia.	Diagramas de secuencia con flujo principal y alternativo (§5).	Modelar interacciones
Flujos alternativos	E2	El prototipo funcional mostraba solo el flujo principal.	El MVP de backend implementa validaciones y excepciones (§6).	Madurez funcional
Repositorio Git	E1/E2	Sin gestión formal del código documentada.	Repositorio con estructura, README y versionamiento (§7).	Gestión técnica

Integración con el MVP y el repositorio. Los artefactos de los Entregables N.º 1 y N.º 2 no aparecen desconectados: las máquinas de estado derivan de las reglas de negocio (RN-02, RN-03, RN-09, RN-10); los diagramas de secuencia ejecutan las historias del MVP (HU-05, HU-09, HU-11); el diagrama integrado es coherente con la arquitectura C4 del Entregable N.º 2; y el código del backend implementa los patrones GoF (Strategy, Observer, State, Repository, Factory Method) y respeta los principios SOLID allí analizados.

3. Máquinas de estado

Se presentan las máquinas de estado UML que representan los ciclos de vida de las dos entidades centrales del MVP. Ambas están alineadas con los requisitos funcionales, las reglas de negocio, las historias de usuario y los flujos implementados, y constituyen la especificación visual del patrón State justificado en el Entregable N.º 2.

3.1 Ciclo de vida de PreRegistroVisita

Entidad modelada. PreRegistroVisita — la solicitud de ingreso de un visitante a la unidad.

La visita nace en estado «Pre-registrada» al crearse (generando su QRToken), y el residente puede aprobarla, rechazarla o cancelarla. Desde «Aprobada», el sistema permite el ingreso solo si el token es válido y la hora está dentro de la ventana de ± 2 horas (regla RN-03); un intento fuera de ventana se registra como denegado sin cambiar el estado. Tras el ingreso, la visita pasa a «Ingresada» y, al registrar la salida, a «Finalizada». Los estados terminales son «Finalizada», «Cancelada» y «Rechazada».

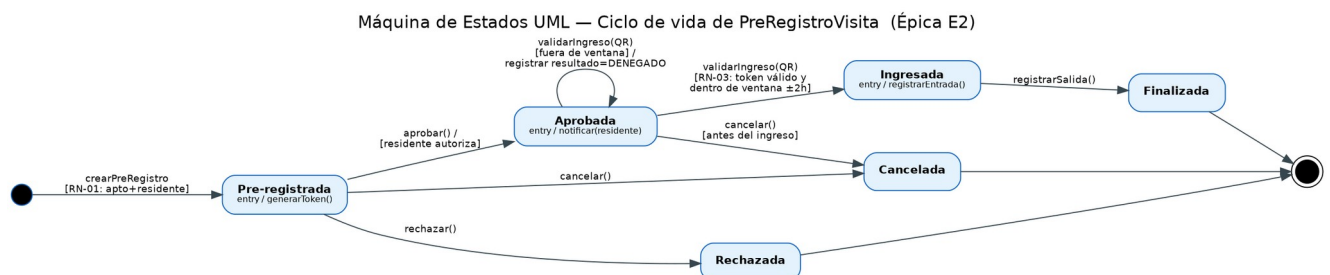


Figura 1. Máquina de estados de PreRegistroVisita (deriva de RN-01, RN-02 y RN-03).

Enlace editable: [\[pegar aquí el enlace al diagrama editable\]](#)

Trazabilidad. Estados según RN-02; guarda de ingreso según RN-03; transición inicial según RN-01 (toda visita asociada a un apartamento y un residente). Historias relacionadas: HU-03, HU-04, HU-05, HU-06.

3.2 Ciclo de vida de PQRS

Entidad modelada. PQRS — petición, queja, reclamo o sugerencia con seguimiento y SLA.

La PQRS nace «Radicada» (con radicado y SLA calculado según prioridad), se «Asigna» a un responsable, entra «En proceso» y, al responderse con evidencia, pasa a «Resuelta». Desde «Resuelta» puede «Cerrarse» (por confirmación o vencimiento) o «Reabrirse» si el residente no queda conforme, regresando a proceso. El cierre exige respuesta final y evidencia, conforme a la regla RN-10.

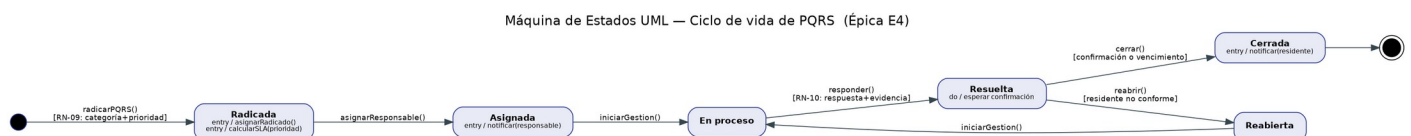


Figura 2. Máquina de estados de PQRS (deriva de RN-09 y RN-10).

Enlace editable: [\[pegar aquí el enlace al diagrama editable\]](#)

Trazabilidad. Cálculo de SLA según RN-09; condición de cierre según RN-10. Historias relacionadas: HU-09, HU-10. El cálculo del vencimiento aplica el patrón Strategy (PoliticaSLA).

4. Diagrama integrado de componentes y distribución

Este diagrama combina la vista de componentes (estructura modular del software) con la vista de despliegue (nodos de ejecución), evitando presentarlas como modelos desconectados. Permite comprender qué componentes conforman el sistema, cómo se comunican, dónde se ejecutan y qué dependencias técnicas requiere la solución. Es coherente con los niveles 2 y 3 de la arquitectura C4 del Entregable N.º 2 y con el código del repositorio.

Diagrama Integrado de Componentes y Despliegue · Residencia360 (MVP)

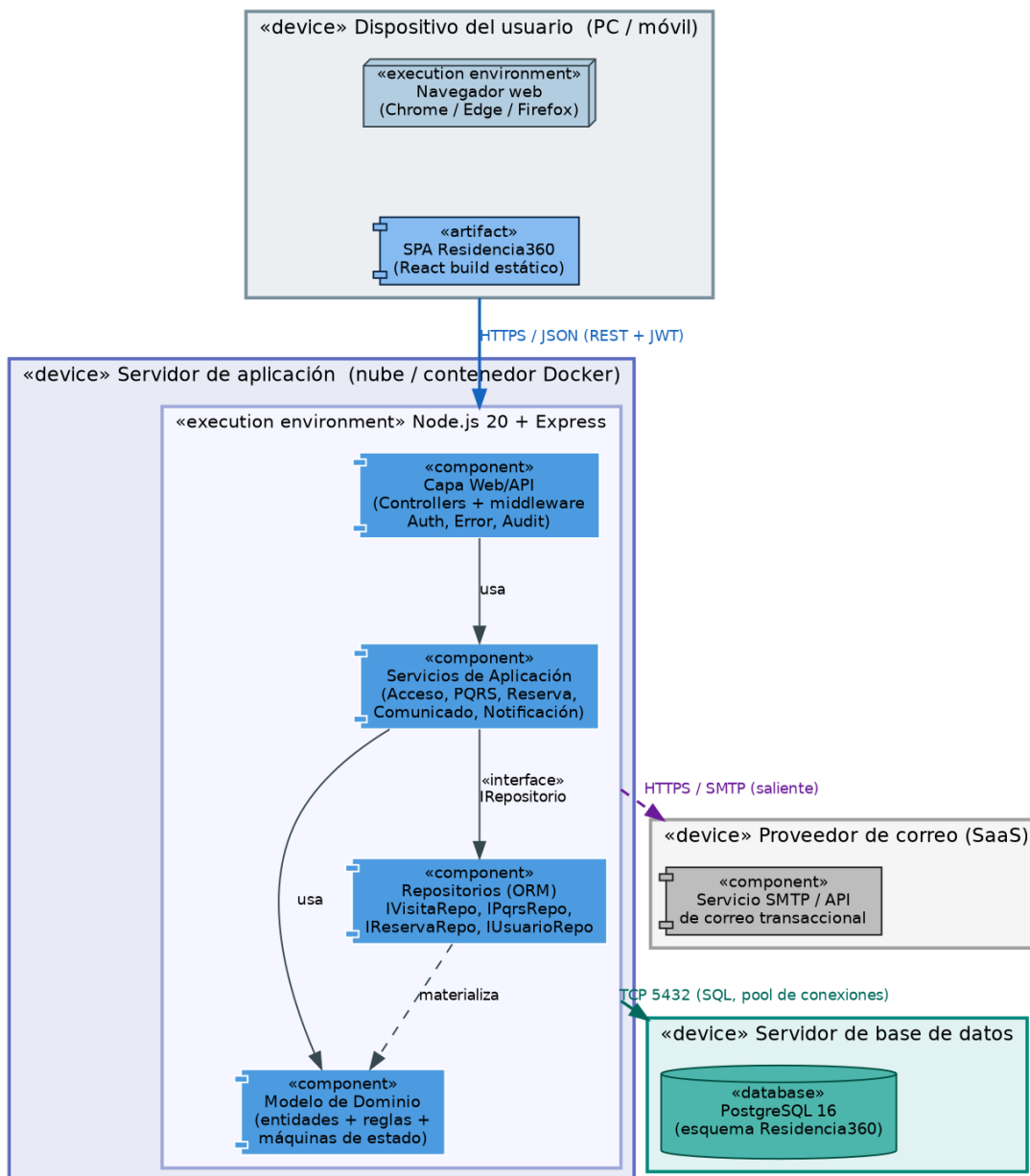


Figura 3. Diagrama integrado de componentes y despliegue de Residencia360 (MVP).

Enlace editable: [\[pegar aquí el enlace al diagrama editable\]](#)

4.1 Responsabilidad de cada componente y nodo

Componente	Responsabilidad	Nodo de ejecución
SPA Residencia360	Interfaz responsive para los cuatro roles; consume la API por HTTPS.	Navegador del usuario
Capa Web/API	Controladores y middleware (autenticación JWT, manejo de errores, auditoría).	Servidor de aplicación (Node.js)
Servicios de aplicación	Orquestan los casos de uso (Acceso, PQRS, Reserva, Comunicado, Notificación).	Servidor de aplicación (Node.js)
Modelo de dominio	Entidades, reglas y máquinas de estado.	Servidor de aplicación (Node.js)
Repositorios (ORM)	Abstracción de persistencia por agregado (patrón Repository).	Servidor de aplicación (Node.js)
Base de datos	Persiste el estado del sistema con integridad referencial.	Servidor de base de datos (PostgreSQL)
Servicio de correo	Entrega de notificaciones por email.	Proveedor SaaS externo

Comunicación entre elementos. El navegador consume la API por HTTPS/JSON con autenticación JWT; la API accede a la base de datos por TCP 5432 mediante un pool de conexiones; y el servicio de notificaciones se comunica con el proveedor de correo por HTTPS/SMTP saliente. Estas conexiones y protocolos quedan explícitos en el diagrama.

Cómo soporta el MVP. La separación en nodos permite escalar y desplegar de forma independiente la SPA, la API y la base de datos; las interfaces de repositorio aíslan el dominio del motor de persistencia; y la frontera de la API concentra la seguridad, soportando los requisitos no funcionales RNF-01 (seguridad) y RNF-04 (rendimiento de las búsquedas en portería).

5. Diagramas de secuencia

Se presentan los diagramas de secuencia UML correspondientes a escenarios del MVP. Cada uno representa cómo interactúan el actor, la interfaz, los controladores, los servicios, las reglas, los repositorios y la base de datos durante la ejecución de un proceso, mostrando el orden temporal de los mensajes, las respuestas del sistema, el flujo principal y los flujos alternativos o de excepción.

Los escenarios se relacionan directamente con las funcionalidades implementadas en el backend (§6).

5.1 HU-05 — Validar ingreso en portería

Actor principal. Vigilante.

Escenario. El vigilante escanea el QR; el sistema busca la visita por su token, valida que el token sea válido y que la hora esté dentro de la ventana, y registra la entrada notificando al residente. El flujo alternativo cubre el token inválido, fuera de ventana o la visita no aprobada, registrando el intento como denegado (RN-03).

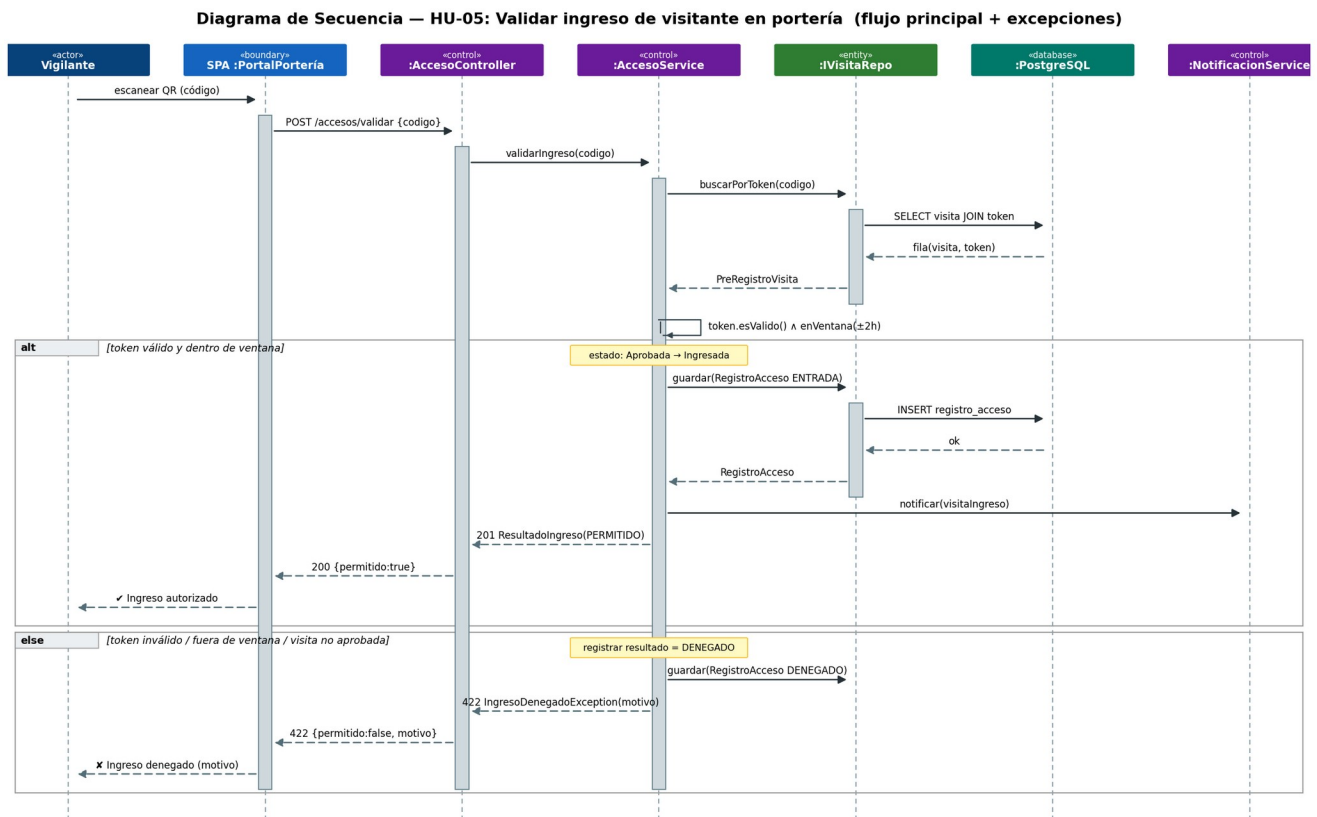


Figura 4. Secuencia HU-05: validar ingreso (flujo principal + excepción).

Enlace editable: [\[pegar aquí el enlace al diagrama editable\]](#)

5.2 HU-09 — Radicar PQRS

Actor principal. Residente.

Escenario. El residente diligencia la PQRS; el controlador valida el DTO; si es válido, el servicio calcula el SLA (patrón Strategy), persiste la PQRS y notifica a administración (patrón Observer). El flujo alternativo cubre el DTO inválido, devolviendo errores de validación.

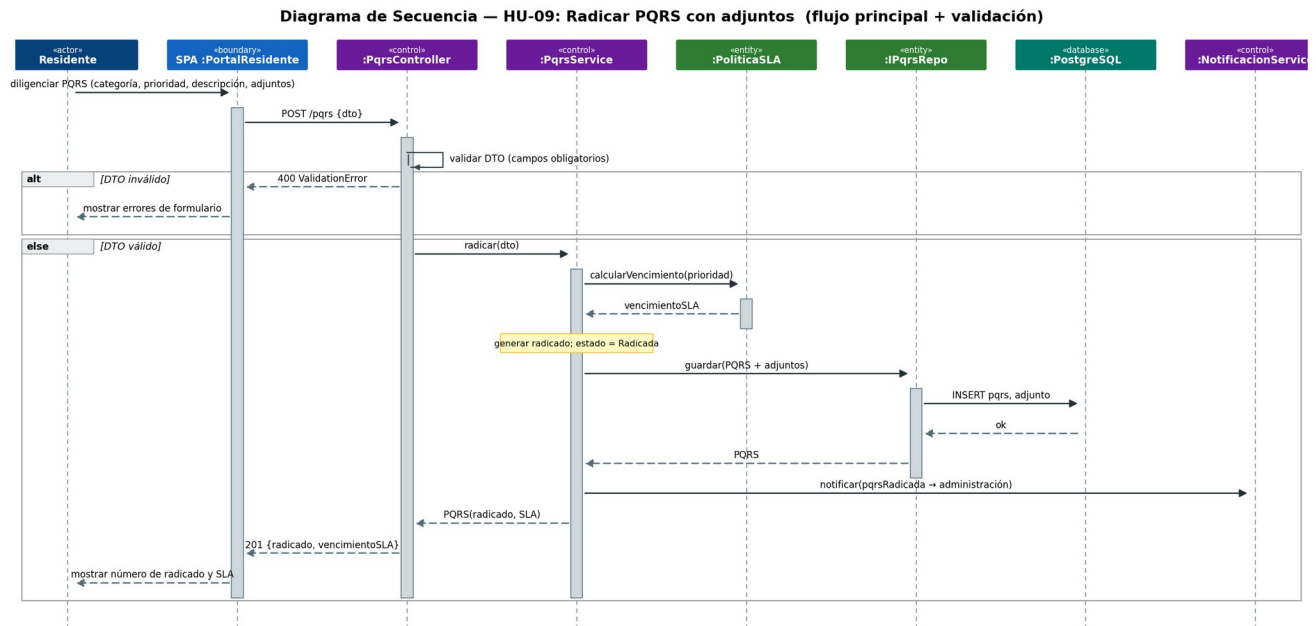


Figura 5. Secuencia HU-09: radicar PQRS (flujo principal + validación).

Enlace editable: [\[pegar aquí el enlace al diagrama editable\]](#)

5.3 HU-11 — Reservar zona común

Actor principal. Residente.

Escenario. El residente solicita una reserva; el servicio verifica que el apartamento no esté en mora (RN-08), busca solapamientos y valida las reglas configurables (RN-06 anticipación, RN-07 máximo semanal). Solo si todo se cumple crea la reserva y notifica. El diagrama muestra los tres flujos alternativos de rechazo anidados.

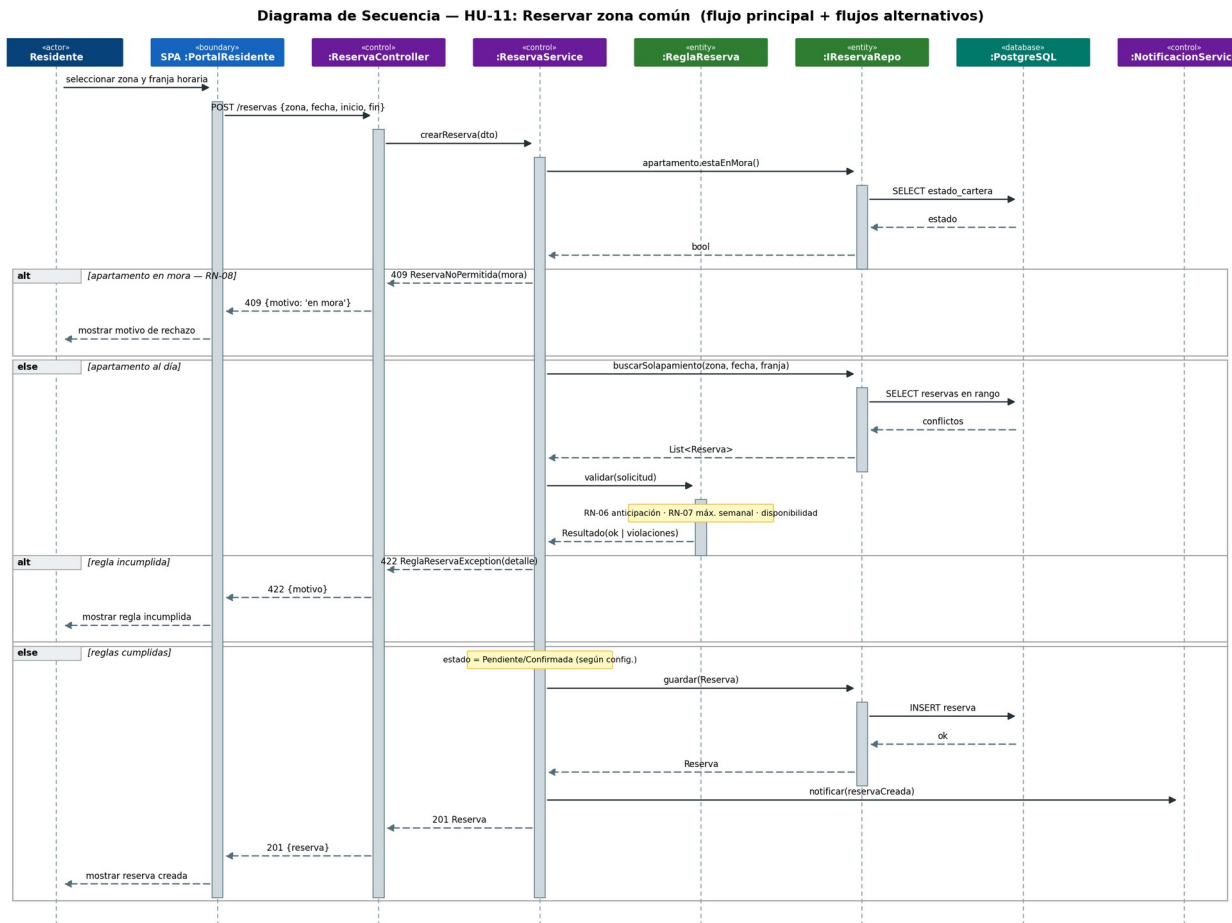


Figura 6. Secuencia HU-11: reservar zona común (flujo principal + flujos alternativos).

Enlace editable: [\[pegar aquí el enlace al diagrama editable\]](#)

6. MVP funcional con flujos alternativos (backend)

A diferencia del Entregable N.º 2 —que mostraba únicamente el flujo principal del prototipo—, en esta entrega el MVP del backend incorpora también los flujos alternativos: validaciones de entrada, manejo de errores y escenarios de excepción. El código aplica buenas prácticas (patrones GoF y principios SOLID) y es coherente con las máquinas de estado, los diagramas de secuencia y el diagrama integrado de componentes.

6.1 Funcionalidades implementadas

- Autenticación y autorización por rol (JWT) — HU-01.
- Pre-registro de visitas, validación de ingreso en portería y registro de salida — HU-03, HU-05, HU-06.
- Radicación y gestión de PQRS con SLA por prioridad — HU-09, HU-10.
- Reserva de zonas comunes con reglas configurables — HU-11, HU-12.
- Publicación de comunicados y notificaciones — HU-07.

6.2 Flujos alternativos, validaciones y manejo de errores

Cada caso de uso implementa, además del flujo principal, los caminos de excepción que las máquinas de estado y los diagramas de secuencia anticipan. El siguiente fragmento ilustra el flujo de validación de ingreso (HU-05) con su manejo de excepciones, coherente con la máquina de estados de §3.1:

```
// AccesoService – flujo principal + alternativo (HU-05)
validarIngreso(codigo) {
  const visita = this.visitaRepo.buscarPorToken(codigo);
  if (!visita) throw new TokenInexistente(); // alt: token no
  existe
  if (!visita.token.esValido()) throw new IngresoDenegado('token'); // alt:
  expirado/usado
  if (!visita.enVentana()) { this.registrarDenegado(visita);
                           throw new IngresoDenegado('ventana'); } // alt: RN-03
  visita.estado.validarIngreso(visita, visita.token); // patrón State: transición legal
  const reg = this.accesoRepo.guardar(RegistroAcceso.entrada(visita));
  this.publicar(new VisitaIngreso(visita)); // patrón Observer: notifica
  return ResultadoIngreso.permitido(reg);
}
```

El manejo centralizado de errores (componente ErrorHandler del nivel 3 del C4) traduce cada excepción de dominio a una respuesta HTTP coherente: TokenInexistente e IngresoDenegado producen un 422 con el motivo; las violaciones de reglas de reserva, un 409 o 422 según el caso. Esto evita estados inválidos y comunica al usuario la causa exacta del rechazo.

6.3 Tecnologías e instrucciones de ejecución

Capa	Tecnología
Frontend	React (SPA responsive) servido como artefacto estático.
Backend / API	Node.js con Express; API REST; autenticación JWT.

Capa	Tecnología
Persistencia	PostgreSQL; acceso mediante repositorios (ORM).
Pruebas	Pruebas de endpoints críticos (RNF-07).

Instrucciones básicas de ejecución. Las instrucciones detalladas (instalación de dependencias, variables de entorno, migraciones de base de datos y arranque del servidor) se documentan en el README del repositorio (§7). Se proveen usuarios de prueba por rol para la demostración.

7. Repositorio Git

El repositorio Git del proyecto evidencia la gestión técnica y colaborativa del código fuente. Incluye el código del MVP, una estructura de carpetas clara, el archivo README, instrucciones de instalación y ejecución, el historial de commits con la participación de ambos integrantes y la documentación mínima del proyecto.

Enlace al repositorio: *[pegar URL del repositorio Git]*

7.1 Estructura de carpetas propuesta

La estructura refleja la arquitectura por capas del modelo C4, lo que facilita localizar cada componente y mantener la coherencia entre el código y los diagramas:

```
residencia360/
├── README.md
├── package.json
├── .env.example
├── src/
│   ├── controllers/      # capa Web/API (AccesoController, PqrsController...)
│   ├── middleware/      # AuthMiddleware, ErrorHandler, AuditInterceptor
│   ├── services/        # servicios de aplicación (casos de uso)
│   ├── domain/          # entidades, estados (State), reglas (Strategy)
│   ├── repositories/    # interfaces + implementaciones ORM (Repository)
│   └── notifications/   # NotificacionService (Observer) + Factory
├── tests/               # pruebas de endpoints críticos
└── docs/                # diagramas y documentación
```

7.2 Contenido mínimo del README

El archivo README.md del repositorio contiene, como mínimo, los siguientes apartados:

- Nombre del proyecto y descripción general.
- Integrantes del equipo.
- Tecnologías utilizadas (React, Node.js/Express, PostgreSQL, JWT).
- Funcionalidades implementadas (mapeadas a las historias de usuario).
- Requisitos de instalación y pasos para ejecutar el proyecto.
- Enlace al despliegue, si aplica, y capturas o evidencias del MVP.

7.3 Buenas prácticas de versionamiento

- Commits descriptivos y atómicos, con participación evidente de ambos integrantes.
- Uso de ramas por funcionalidad y fusión mediante solicitudes de cambios.
- Archivo .gitignore y .env.example para no versionar credenciales.
- Archivo de dependencias (package.json) y, si aplica, scripts de migración.

8. Anexos y referencias

8.1 Anexos (soportes del trabajo)

Soporte	Enlace / Ubicación
Repositorio Git	[pegar URL del repositorio Git]
Prototipo Figma (origen)	https://rare-jeep-73888365.figma.site/login
Máquinas de estado (editables)	[pegar enlaces editables]
Diagramas de secuencia (editables)	[pegar enlaces editables]
Diagrama integrado componentes/despliegue	[pegar enlace editable]
Capturas y evidencia de ejecución del MVP	[adjuntar capturas]
Evidencia de commits	[captura del historial de commits]

8.2 Referencias

- Gamma, E., Helm, R., Johnson, R. y Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.
- Martin, R. C. (2017). Clean Architecture. Prentice Hall.
- Brown, S. (2023). The C4 model for visualising software architecture. c4model.com.
- Object Management Group (2017). Unified Modeling Language (UML), versión 2.5.1.
- Fowler, M. (2002). Patterns of Enterprise Application Architecture. Addison-Wesley.